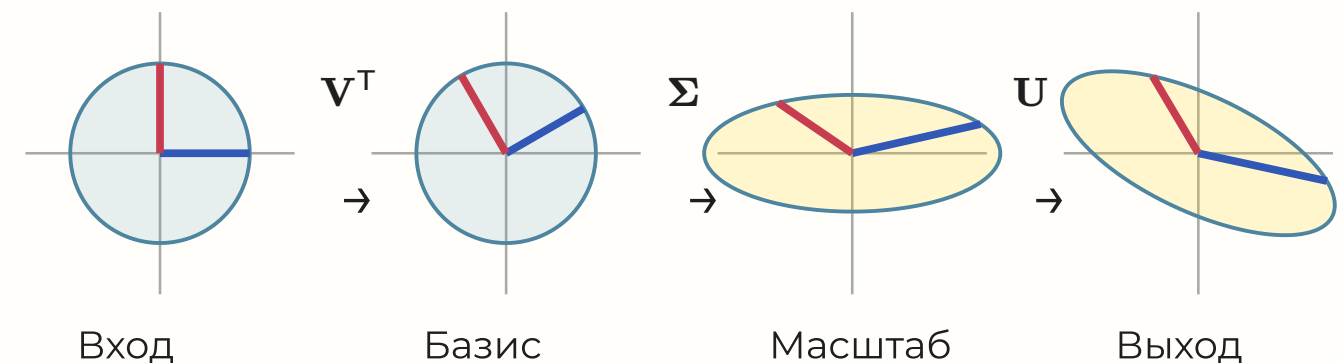


SVD, PCA и честная оценка модели

Как устроить представление данных – и проверить, что оно помогает?

МФТИ · Машинное обучение

5 октября 2026



Что осталось с прошлой лекции?

Поворот сохраняет
геометрию



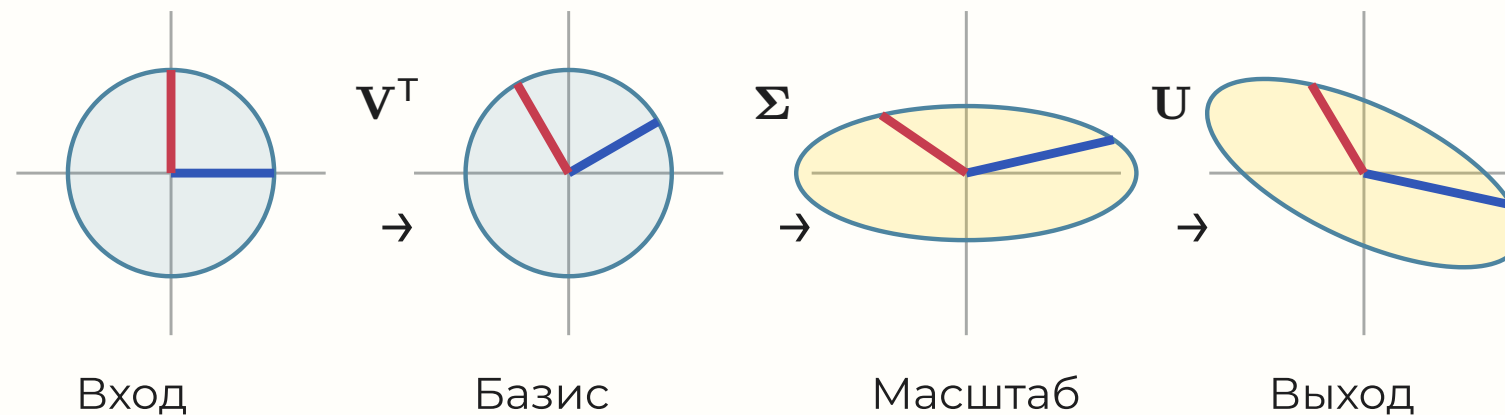
РСА оставляет часть
направлений



SVD даёт оси и масштабы

Сегодня: вычислить разложение, выбрать число компонент и проверить предсказания.

Что делает матрица с направлениями?



$$Ax = U\Sigma V^T x$$

Сначала сменить входной базис.
Затем растянуть оси. Затем
сменить выходной базис.

Ортогональное преобразование может
включать отражение.

Сингулярное разложение: три множителя

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$$

$$\mathbf{U} \in \mathbb{R}^{m \times r}$$

выходные направления

$$\mathbf{\Sigma} \in \mathbb{R}^{r \times r}$$

неотрицательные масштабы

$$\mathbf{V} \in \mathbb{R}^{d \times r}$$

входные направления

$$\mathbf{U}^T \mathbf{U} = \mathbf{I}_r, \quad \mathbf{V}^T \mathbf{V} = \mathbf{I}_r, \quad \sigma_1 \geq \dots \geq \sigma_r > 0$$

Матрица $\mathbf{A} \in \mathbb{R}^{m \times d}$; r – число независимых направлений (ранг). $\mathbf{I}_r$ – единичная матрица. Компактная SVD опускает нулевые направления.

Откуда взять входные направления?

$$\mathbf{A}^T \mathbf{A} \mathbf{v}_j = \lambda_j \mathbf{v}_j$$

$\mathbf{A}^T \mathbf{A}$ неотрицательно
определена

→

$$\lambda_j \geq 0$$

→

$$\sigma_j = \sqrt{\lambda_j}$$

$$\mathbf{u}_j = \frac{\mathbf{A} \mathbf{v}_j}{\sigma_j} \quad (\sigma_j > 0)$$

Собственные векторы выбрать ортонормальными; одинаковые значения допускают разные базисы.

Посчитаем на одной матрице

$$\mathbf{A} = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}$$

$$\mathbf{A}^T \mathbf{A} = \begin{bmatrix} 10 & 6 \\ 6 & 10 \end{bmatrix}$$

$$\lambda_1 = 16, \quad \lambda_2 = 4$$

$$\mathbf{v}_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \mathbf{v}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

$$\sigma_1 = 4, \quad \sigma_2 = 2$$

Теперь найти выходные направления

$$\mathbf{A}\mathbf{v}_1 = 4\mathbf{v}_1, \quad \mathbf{A}\mathbf{v}_2 = 2\mathbf{v}_2$$

$$\mathbf{U} = \mathbf{V} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad \mathbf{\Sigma} = \begin{bmatrix} 4 & 0 \\ 0 & 2 \end{bmatrix}$$

$$\mathbf{U}\mathbf{\Sigma}\mathbf{V}^T = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}$$

Равенство $\mathbf{U} = \mathbf{V}$ здесь – свойство выбранной симметричной положительно определённой матрицы.

Один ортогональный шаг: обнулить элемент

$$\mathbf{A} = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}$$

$$\mathbf{G} = \frac{1}{\sqrt{10}} \begin{bmatrix} 3 & 1 \\ -1 & 3 \end{bmatrix}$$

$$\mathbf{G}^T \mathbf{G} = \mathbf{I}_2$$

$$\mathbf{GA} = \begin{bmatrix} \sqrt{10} & 6/\sqrt{10} \\ 0 & 8/\sqrt{10} \end{bmatrix}$$

Обнулить нижний элемент первого столбца, сохранив длины.

Для большой матрицы аналогично действуют слева и справа; затем остаётся bidiagonalная задача.

Как SVD считает компьютер?

Ортогональные
преобразования

→ Бидиагональная матрица →

Итерационное
разложение

$$\mathbf{A} = \mathbf{L}\mathbf{B}\mathbf{W}^T, \quad \mathbf{B} = \mathbf{U}_B \mathbf{\Sigma} \mathbf{V}_B^T$$

$$\mathbf{U} = \mathbf{L}\mathbf{U}_B, \quad \mathbf{V} = \mathbf{W}\mathbf{V}_B$$

Здесь полные множители: $\mathbf{L} \in \mathbb{R}^{m \times m}$, $\mathbf{W} \in \mathbb{R}^{d \times d}$ ортогональны; $\mathbf{B} \in \mathbb{R}^{m \times d}$ бидиагональна.

Матрица $\mathbf{A}^T \mathbf{A}$ удобна для ручного вывода; численный алгоритм может обходиться без неё.

Что потеряется, если оставить один масштаб?

$$\mathbf{A} = 4\mathbf{u}_1\mathbf{v}_1^T + 2\mathbf{u}_2\mathbf{v}_2^T$$

$$\mathbf{A}_1 = 4\mathbf{u}_1\mathbf{v}_1^T = \begin{bmatrix} 2 & 2 \\ 2 & 2 \end{bmatrix}$$

$$\|\mathbf{A} - \mathbf{A}_1\|_F^2 = 4$$

Ранг – число независимых направлений.

Ранг 1: все выходы лежат на одной прямой.

Норма Фробениуса: квадрат – сумма квадратов всех элементов.

Почему отбрасывать меньшие значения?

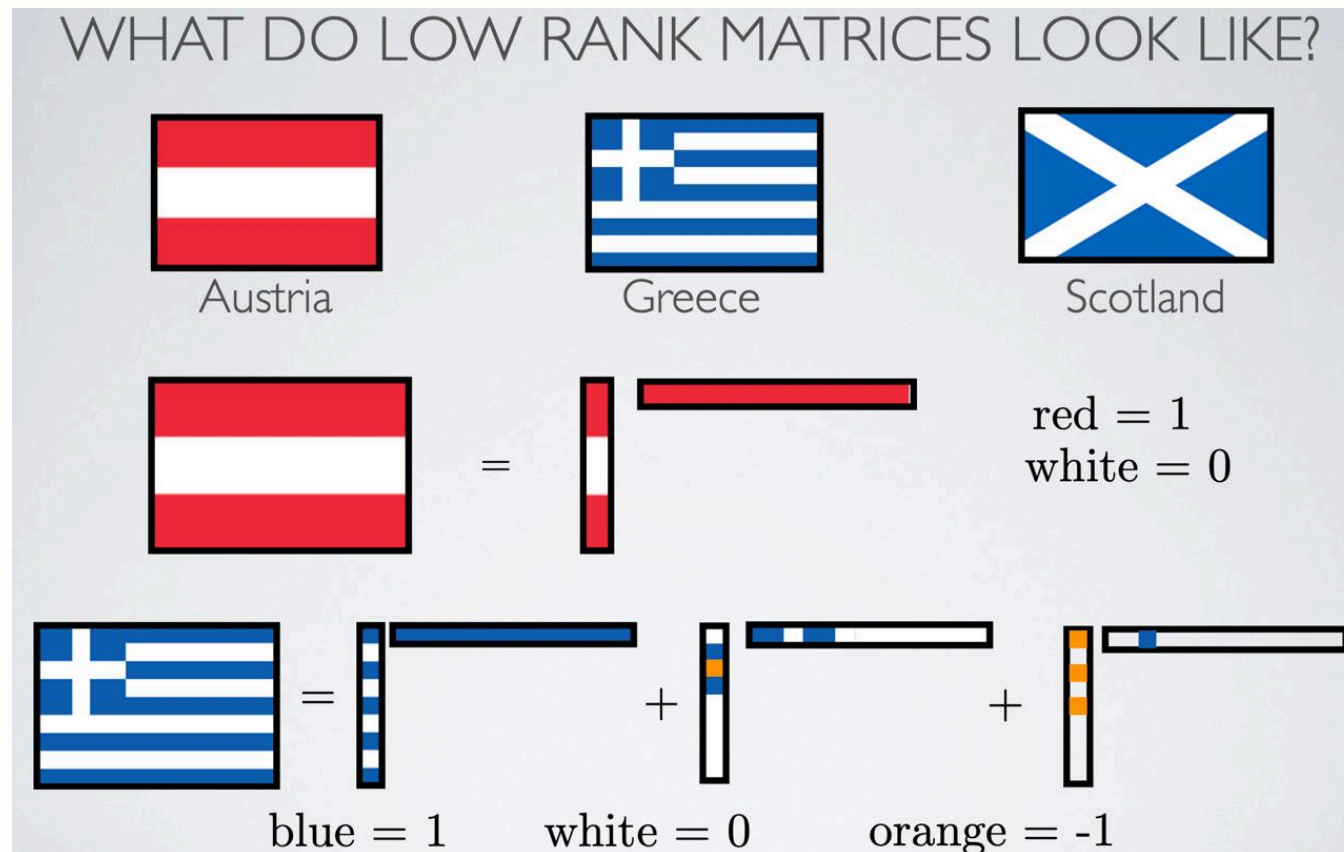
$$\mathbf{A}_k = \sum_{j=1}^k \sigma_j \mathbf{u}_j \mathbf{v}_j^T$$

$$\min_{\text{rank}(\mathbf{B}) \leq k} \|\mathbf{A} - \mathbf{B}\|_F^2 = \|\mathbf{A} - \mathbf{A}_k\|_F^2 = \sum_{j>k} \sigma_j^2$$

Лучшее приближение заданного ранга по сумме квадратов ошибок.

$0 \leq k \leq r$; k – число сохранённых направлений. Критерий – восстановление матрицы.

Низкий ранг виден даже у картинки



Одноцветная вертикальная полоса:
столбец × строка.

$$\mathbf{ab}^T \Rightarrow \text{rank} \leq 1$$

Ранг суммы может быть больше.

Австрия: красный 1, белый 0. Греция: синий 1, белый 0, оранжевый -1 в разложении.

Архивный пример: флаги как суммы простых матриц

Проверка: что сохраняет усечение?

Матрица $\mathbf{A} = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}$ получила только первую компоненту.

Что произойдёт с векторами $(1, 1)^T$ и $(1, -1)^T$?

$$\mathbf{A}_1 \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 4 \end{bmatrix}, \quad \mathbf{A}_1 \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \mathbf{0}$$

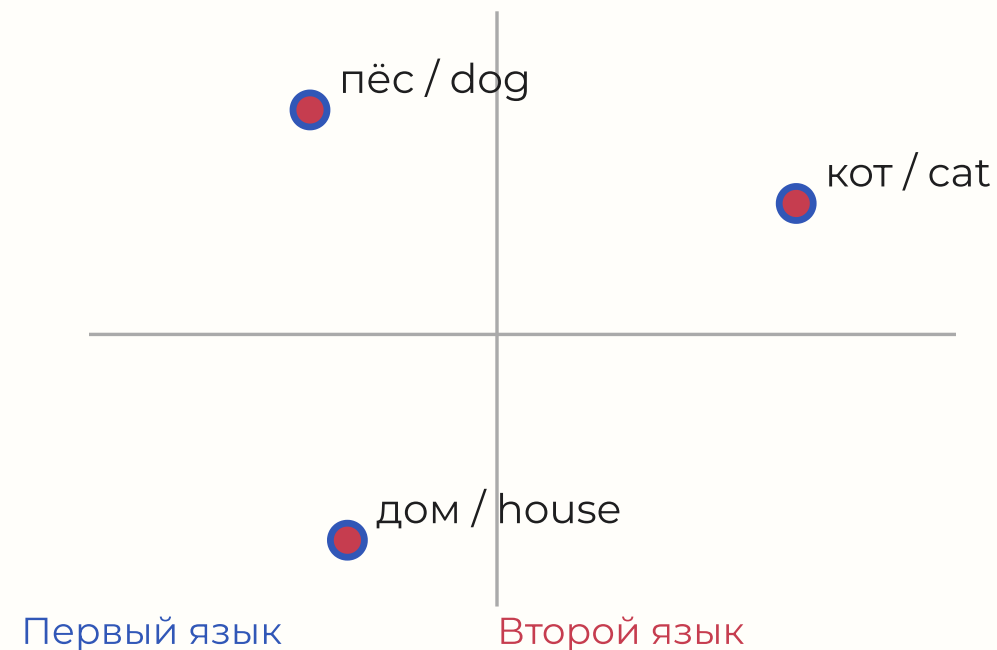
Первое направление сохранено; второе уничтожено.

Как совместить два облака слов?

Соответствующие слова →

Одна ортогональная матрица

→ Близкие парные векторы



Известные пары задают соответствия.

Поворот сохраняет длины и расстояния.

Вернуть исходное

Учебные координаты, не реальные эмбединги.

Какая матрица совместит известные пары?

Строки $\mathbf{Q}, \mathbf{K} \in \mathbb{R}^{a \times d}$ – соответствующие векторы; a – число пар.

$$\min_{\mathbf{R}^T \mathbf{R} = \mathbf{I}_d} \|\mathbf{Q}\mathbf{R} - \mathbf{K}\|_F^2$$

$$\|\mathbf{Q}\mathbf{R} - \mathbf{K}\|_F^2 = \|\mathbf{Q}\|_F^2 + \|\mathbf{K}\|_F^2 - 2 \operatorname{tr}(\mathbf{R}^T \mathbf{Q}^T \mathbf{K})$$

След tr – сумма диагональных элементов. Минимизировать ошибку = максимизировать последний след.

SVD решает и эту задачу

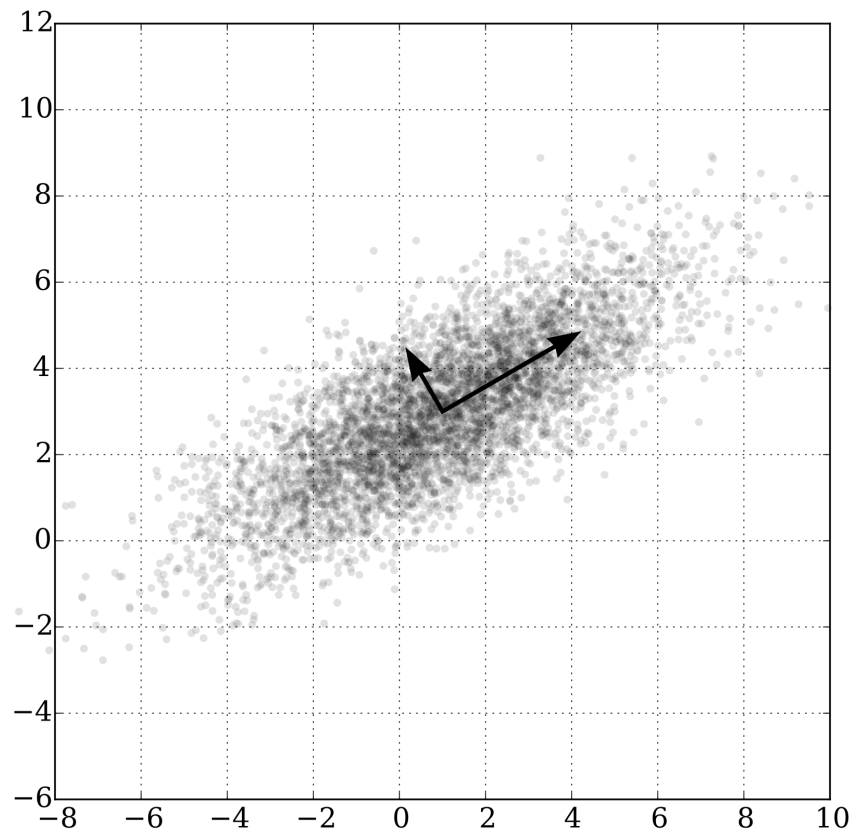
$$\mathbf{H} = \mathbf{Q}^T \mathbf{K} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T$$

$$\mathbf{R}_* = \mathbf{U} \mathbf{V}^T$$

Масштабы отброшены: осталось ортогональное отображение.

Если нужен только поворот, добавить $\det \mathbf{R} = 1$; при необходимости изменить знак последнего направления.

Какую прямую выбрать для проекции?



Исходная схема PCA: два направления одного облака

PCA – метод главных компонент.

Первая ось сохраняет наибольший разброс проекций относительно центра.

Следующие оси ортогональны предыдущим.

У данных есть строки, у объектов – столбцы

$$\mathbf{x}^{(i)} \in \mathbb{R}^d$$

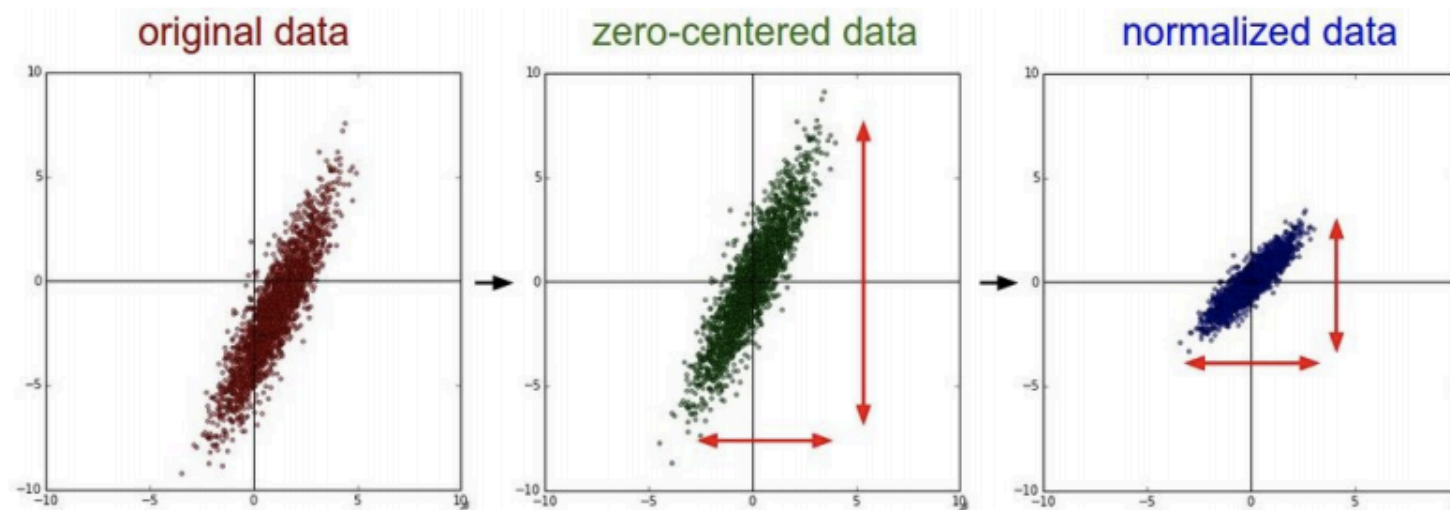
n наблюдений; d признаков.

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}^{(1)\top} \\ \vdots \\ \mathbf{x}^{(n)\top} \end{bmatrix} \in \mathbb{R}^{n \times d}$$

$$\boldsymbol{\mu} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}^{(i)}$$

$$\mathbf{X}_c = \mathbf{X} - \mathbf{1}_n \boldsymbol{\mu}^\top$$

Центрирование $\neq$ масштабирование



Исходное облако $\rightarrow$ центрирование $\rightarrow$ дополнительное масштабирование

Для обычной PCA центрировать признаки. Масштабирование выбирать по единицам и задаче.

Почему первая ось совпадает с SVD?

$$\mathbf{v}_1 = \arg \max_{\|\mathbf{v}\|_2=1} \frac{\|\mathbf{X}_c \mathbf{v}\|_2^2}{n-1}$$

$$\mathbf{C} = \frac{\mathbf{X}_c^\top \mathbf{X}_c}{n-1}, \quad \mathbf{C} \mathbf{v}_j = \frac{\sigma_j^2}{n-1} \mathbf{v}_j$$

Ковариационная матрица $\mathbf{C}$ показывает совместный разброс признаков; $n > 1$.

Правые сингулярные векторы $\mathbf{X}_c$ дают оси PCA.

Разложение того же набора точек

$$\mathbf{X}_c = \begin{bmatrix} 3 & 1 \\ 1 & 3 \\ -3 & -1 \\ -1 & -3 \end{bmatrix}$$

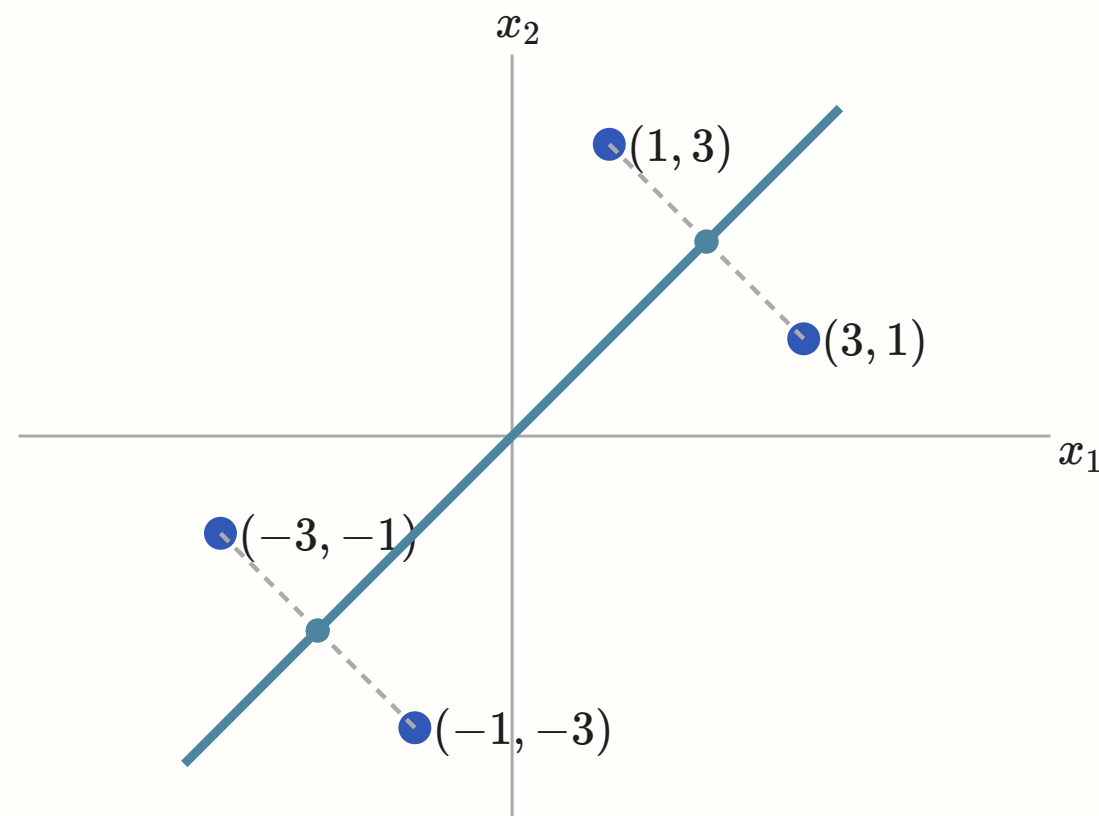
$$\mathbf{U} = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & -1 \\ -1 & -1 \\ -1 & 1 \end{bmatrix}$$

$$\mathbf{\Sigma} = \begin{bmatrix} 4\sqrt{2} & 0 \\ 0 & 2\sqrt{2} \end{bmatrix}$$

$$\mathbf{V}^T = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

$$\mathbf{X}_c : 4 \times 2; \mathbf{U} : 4 \times 2; \mathbf{\Sigma}, \mathbf{V} : 2 \times 2.$$

Повернуть ось: сколько разброса осталось?



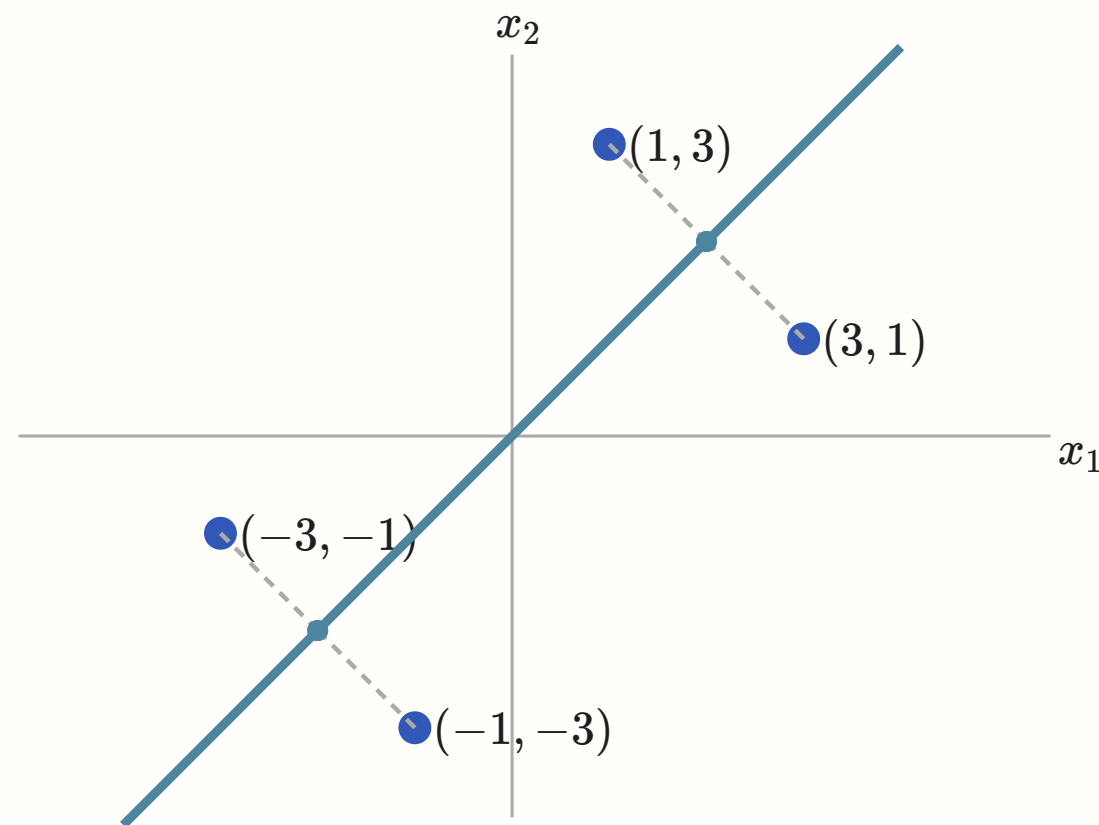
Угол оси 45°

Синие точки – наблюдения. Бирюзовые – проекции; z_i – координата проекции.

$$\sum_i z_i^2 = 32,0 \quad (80\%)$$

Первая ось: $\mathbf{v}_1 = (1, 1)^T / \sqrt{2}$.

Новая координата и обратная проекция



$$\mathbf{v}_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \mathbf{x}^{(1)} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$$

$$z_1 = \mathbf{v}_1^T \mathbf{x}^{(1)} = 2\sqrt{2}$$

$$\hat{\mathbf{x}}^{(1)} = z_1 \mathbf{v}_1 = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$$

Отклонение $(1, -1)^T$ потеряно.

Оси, координаты и восстановление

$$\mathbf{X}_c = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$$

$$\mathbf{Z} = \mathbf{X}_c \mathbf{V}_k = \mathbf{U}_k \mathbf{\Sigma}_k \in \mathbb{R}^{n \times k}$$

$$\hat{\mathbf{X}} = \mathbf{Z} \mathbf{V}_k^\top + \mathbf{1}_n \boldsymbol{\mu}^\top$$

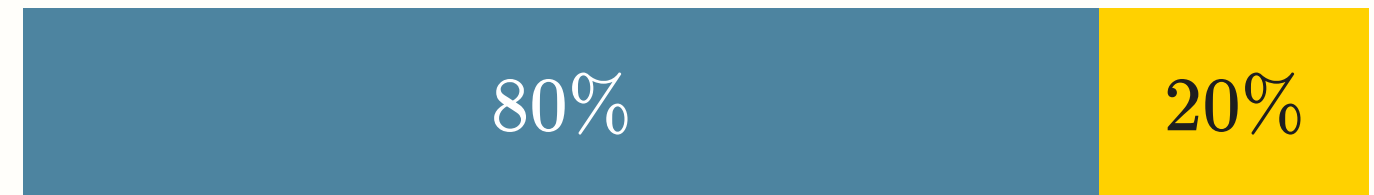
$\mathbf{V}_k \in \mathbb{R}^{d \times k}$ – оси; $\mathbf{Z}$ – новые координаты объектов.

СКОЛЬКО КОМПОНЕНТ ОСТАВИТЬ?

$$\text{EVR}_j = \frac{\sigma_j^2}{\sum_{l=1}^r \sigma_l^2}$$

$$\sigma_1^2 = 32, \quad \sigma_2^2 = 8$$

$$\text{EVR}_1 = \frac{32}{40} = 0,8$$



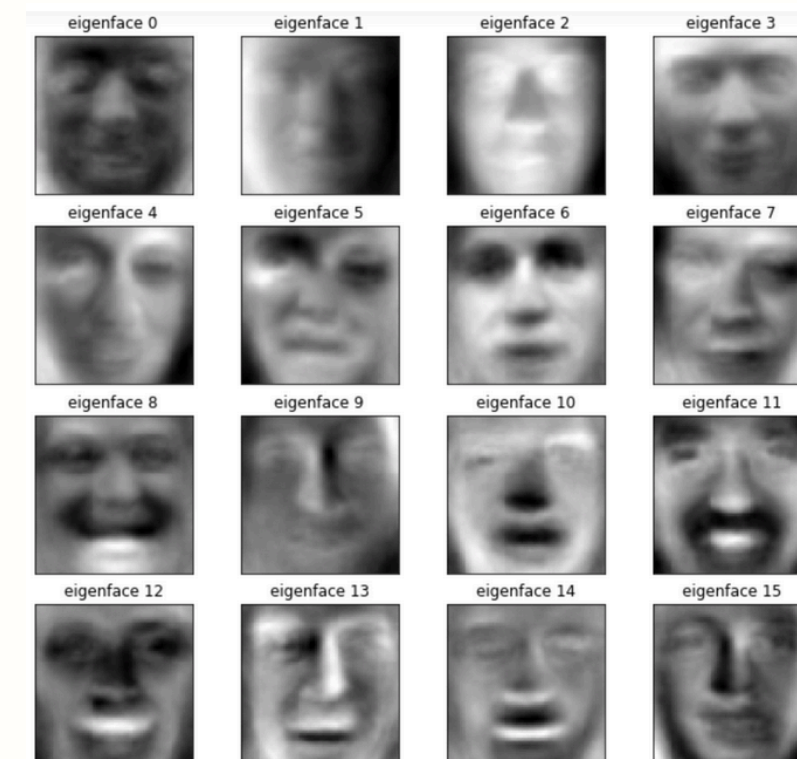
В нашем примере одна компонента сохраняет 80% суммарной дисперсии.

Порог дисперсии – правило сжатия, а не гарантия полезного предсказания.

Компоненты можно увидеть



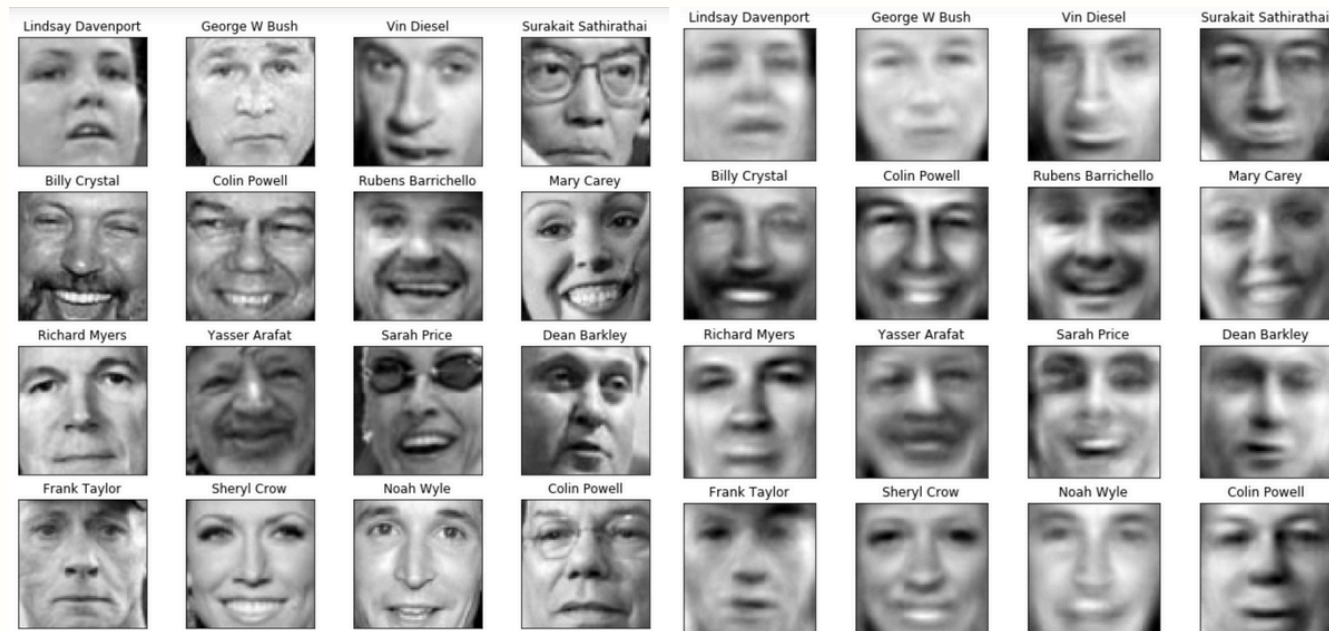
Архивный набор лиц



Те же источники: 16 базовых компонент

Один объект описывается весами общих компонент; компонента сама может выглядеть как размытое лицо.

Сколько деталей вернётся?



Оригиналы и восстановление: 50 компонент



Оригиналы и восстановление: 250 компонент

В каждом блоке слева – оригиналы, справа – восстановление.

Большой разброс может быть бесполезным для ответа

Представьте: один признак сильно меняется, но не связан с классом. Другой меняется мало, но разделяет классы.

РСА выбирает
дисперсию



Классификатор
использует метки



Проверка измеряет
полезность

Число компонент k для предсказания выбирать по качеству на новых данных.

На каких данных выбирать число компонент?

Обучение: подобрать
параметры



Валидация: выбрать k



Тест: оценить итог

Метрика – численный показатель качества прогнозов.

Если тест меняет решение о модели, он уже участвует в выборе.

Что считается новым объектом?

Случайное разбиение: независимые новые наблюдения.

Группы: целый человек или источник в одной части.

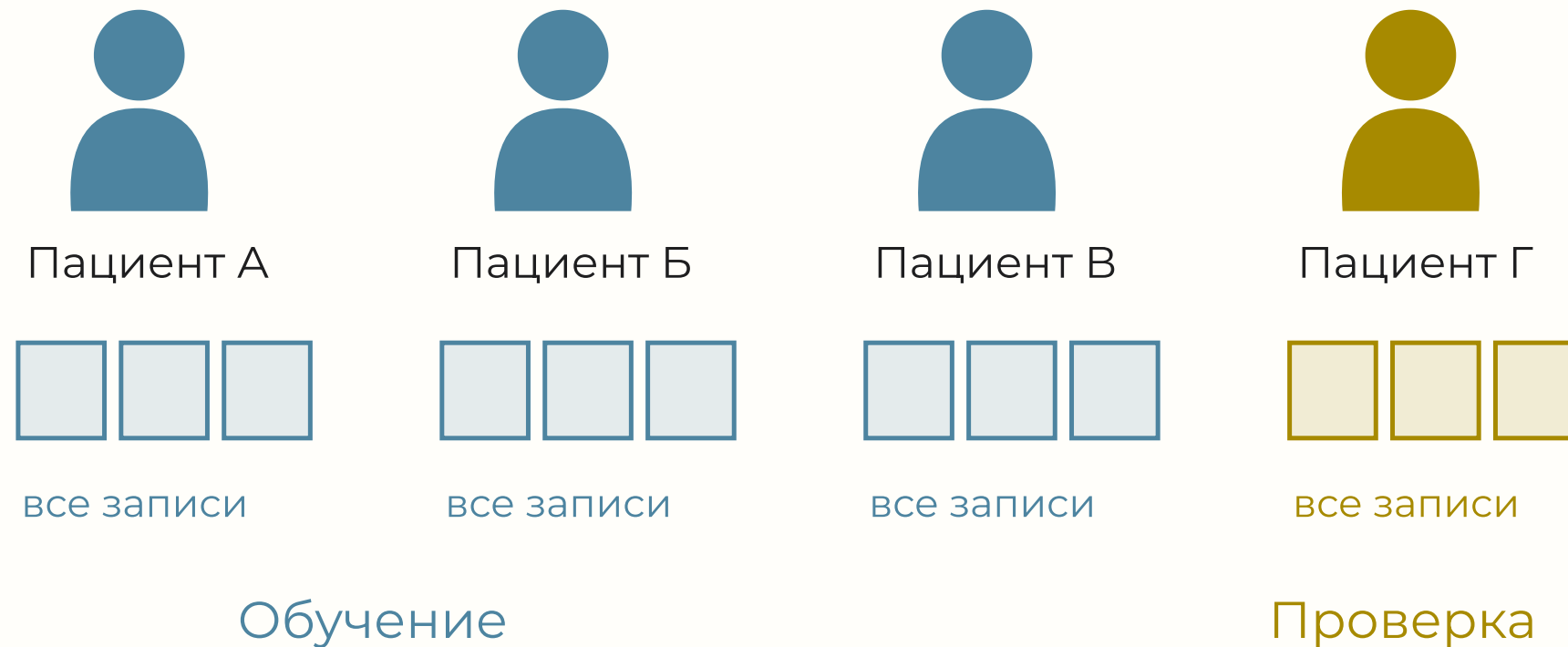
Время: учиться на прошлом, проверять на будущем.

прошлое: обучение

будущее:
проверка

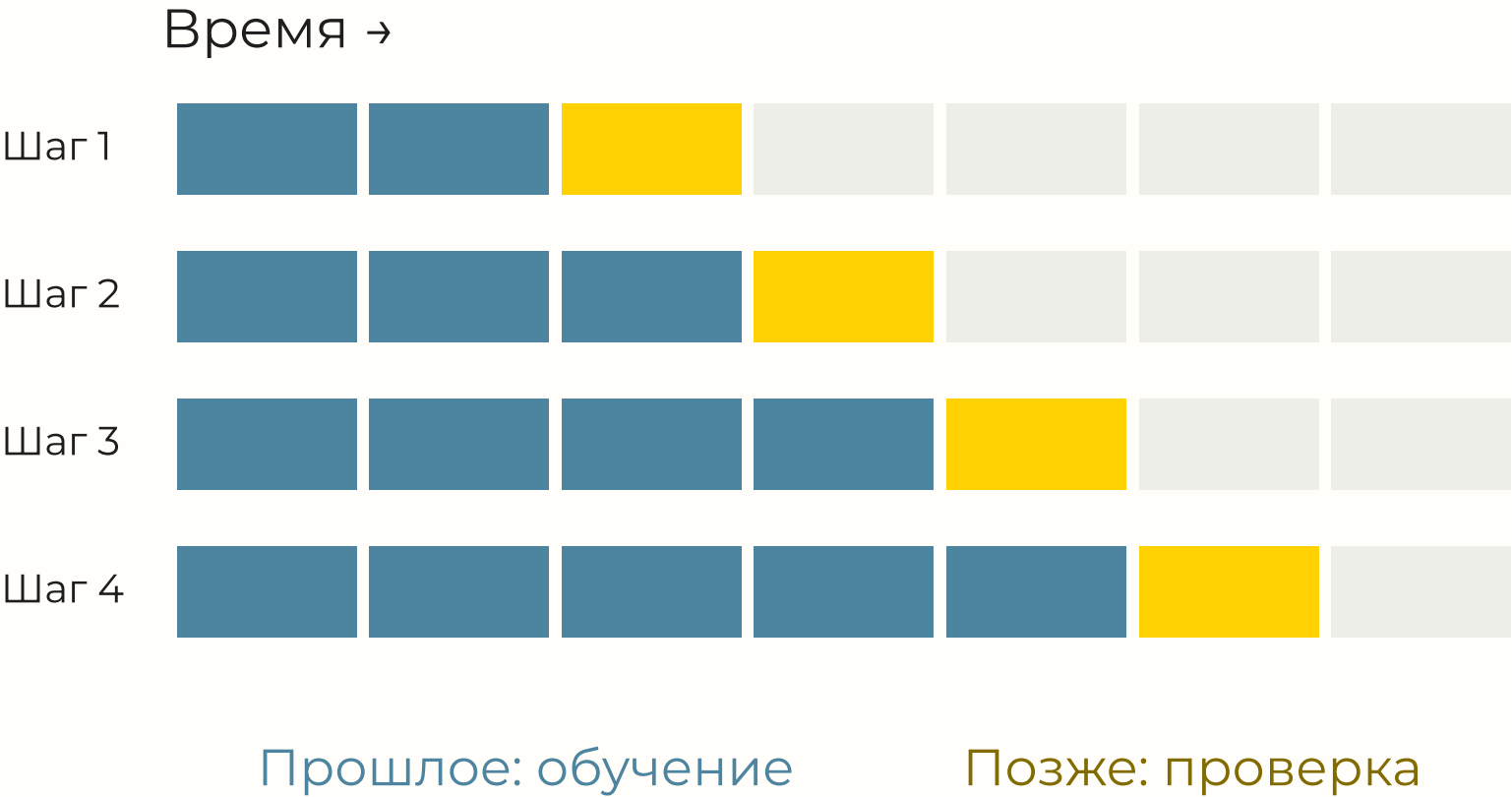
Стратификация сохраняет доли классов, но сама по себе не устраняет зависимость между объектами.

Все записи пациента оставить вместе



Если нужен прогноз для нового пациента, все его записи оставить для проверки.

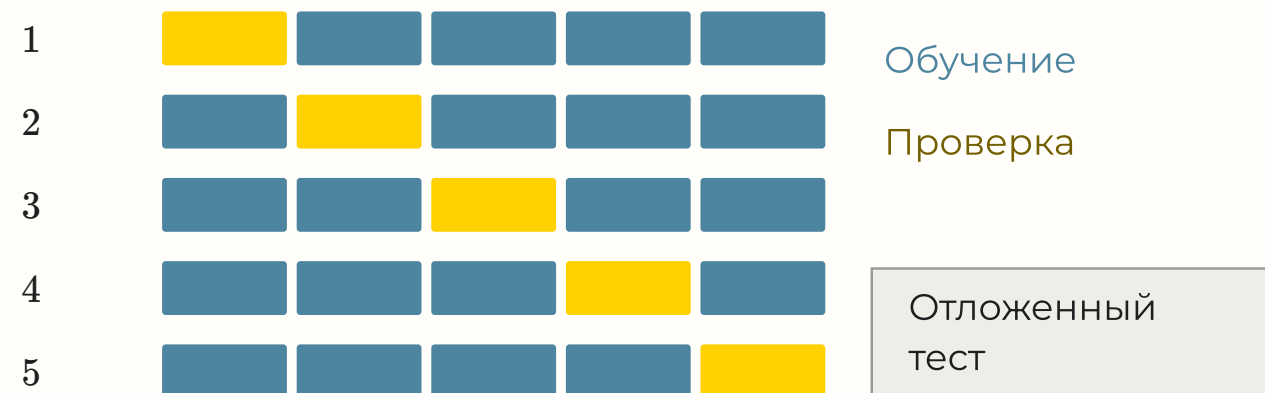
Обучаться на прошлом, проверять на будущем



Проверочное окно позже обучающего. Порядок времени сохранить внутри CV.

Кросс-валидация повторяет обучение и проверку

Обучающий набор: повторное разбиение



Разбиение меняется; правило проверки сохраняется.

Разделить обучающий набор на F частей – фолдов.

Каждый фолд по очереди служит валидацией; остальные – обучением.

$$\overline{M} = \frac{1}{F} \sum_{f=1}^F M_f$$

M_f – метрика на проверочном фолде f . Тест остаётся вне цикла.

Центр и оси PCA обучать внутри каждого фолда

Обучающая часть фолда

Оценить μ и масштабы →

Обучить PCA →

Обучить классификатор

Валидационная часть фолда

Применить те же параметры →

Получить прогноз →

Посчитать метрику

Предобработка всей выборки до CV даёт валидационным объектам влияние на обученную модель.

Выбрать модель, затем один раз открыть тест

Сравнить
варианты по CV



Зафиксировать k ,
модель и порог



Обучить на
доступном train



Оценить на тесте

После выбора переобучить всю цепочку: предобработка → PCA → классификатор.

CV для выбора и CV для итоговой оценки – разные роли. Для честной оценки всего выбора нужна внешняя проверка.

Где здесь утечка?

1. Найти среднее и оси PCA по всем данным.
2. Разбить преобразованные данные на фолды.
3. Выбрать k по средней точности.

Утечка начинается в шаге 1: проверочная часть повлияла на представление.

Сначала разбить; затем в каждом фолде заново обучить предобработку.

Что значит «хорошо предсказать»?

$y^{(i)}$ – истинный класс; $\hat{y}^{(i)}$ – предсказанный.

Объект	1	2	3	4	5	6	7	8	9	10
$y^{(i)}$	1	0	1	0	0	0	1	0	0	0
$\hat{y}^{(i)}$	0	0	1	0	0	0	1	0	1	0

$$\text{Accuracy} = \frac{1}{n} \sum_{i=1}^n \mathbf{1}[\hat{y}^{(i)} = y^{(i)}] = \frac{8}{10} = 0,8$$

$\mathbf{1}[\cdot]$ равно 1, если условие верно, иначе 0. Какие две ошибки допущены?

Разделить ошибки по их направлению

	Предсказан 1	Предсказан 0
Истинный 1	TP = 2	FN = 1
Истинный 0	FP = 1	TN = 6

Положительный класс – $y = 1$. TP, TN – верные ответы; FP, FN – два типа ошибки.

Высокая ассурасу может скрыть провал

Учебный набор: 990 отрицательных и 10 положительных объектов.

Предсказывать всем 0.

$$\text{Accuracy} = 0,99$$

$$TP = 0, \quad FN = 10$$

$$\text{Recall} = \frac{TP}{TP + FN} = 0$$

Ни один положительный объект не найден.

Два вопроса – два знаменателя

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{2}{3}$$

Среди найденных положительных – сколько действительно положительных?

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{2}{3}$$

Среди всех положительных – сколько удалось найти?

При нулевом знаменателе значение не определено; правило обработки задаётся отдельно.

Если нужен один показатель

$$F_1 = \frac{2 \text{ Precision Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN}$$

$$F_1 = \frac{4}{4 + 1 + 1} = \frac{2}{3}$$

Гармоническое среднее: обе величины должны быть высокими.

F1 не учитывает TN и сам по себе не задаёт стоимость ошибок.

Функция потерь и метрика отвечают разным задачам

На той же задаче обучать вероятности $p^{(i)}$ по log loss:

$$Q = -\frac{1}{n} \sum_i \left[y^{(i)} \log p^{(i)} + (1 - y^{(i)}) \log(1 - p^{(i)}) \right]$$

$p^{(i)} = P(y = 1 \mid \mathbf{x}^{(i)})$ по модели; $0 < p^{(i)} < 1$.

Выбирать классы по порогу t :

$$\hat{y}^{(i)} = \mathbf{1}[p^{(i)} \geq t]$$

Accuracy/F1 считают уже пороговые решения.

Log loss может служить и метрикой на валидации.

Один скор – разные решения

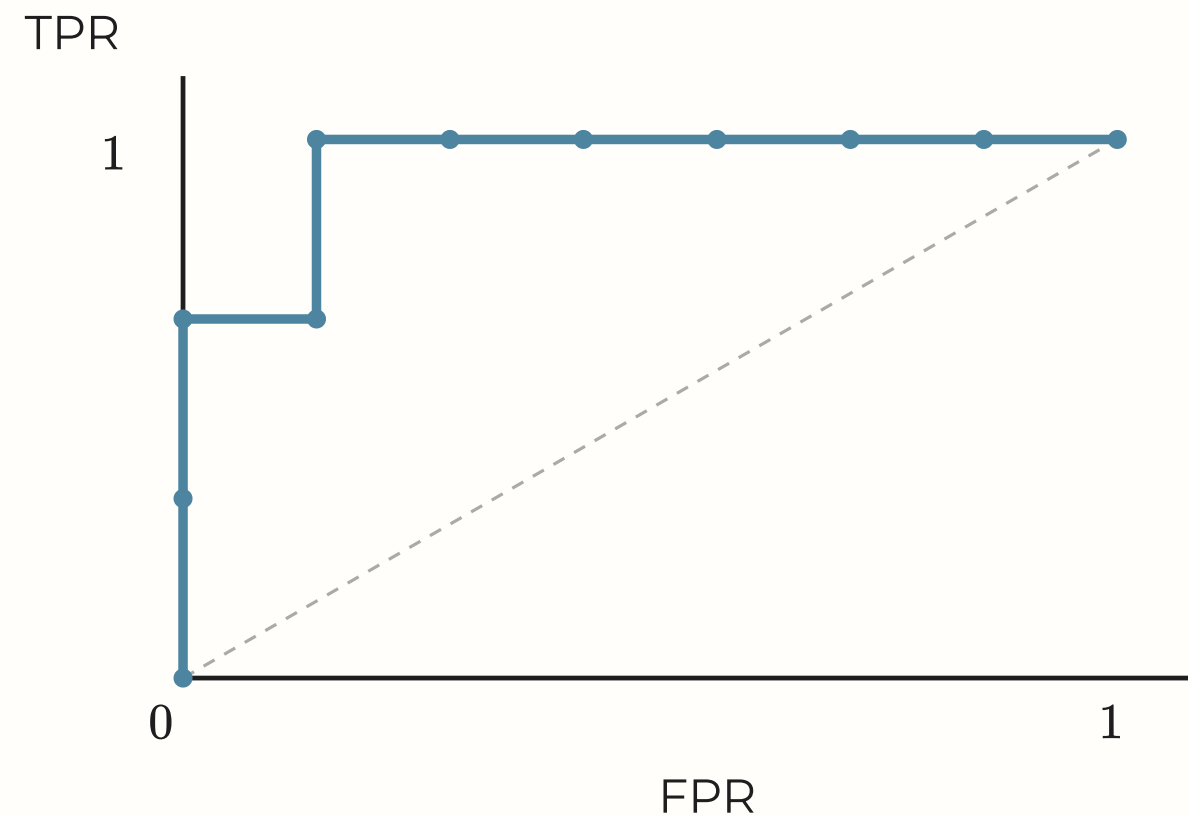
Скор $s^{(i)}$ упорядочивает объекты; положительный класс присвоить при $s^{(i)} \geq t$.

0,9	0,8	0,6	0,4	0,35	0,3	0,25	0,2	0,15	0,1
$y = 1$	$y = 1$	$y = 0$	$y = 1$	$y = 0$	$y = 0$	$y = 0$	$y = 0$	$y = 0$	$y = 0$

Порог $TP = 2, FP = 1, FN = 1, TN = 6$
Precision = 0,67, Recall = 0,67

Граница меняет Precision и Recall. Выбирать её на валидации, исходя из задачи.

ROC: что меняется вместе с порогом?



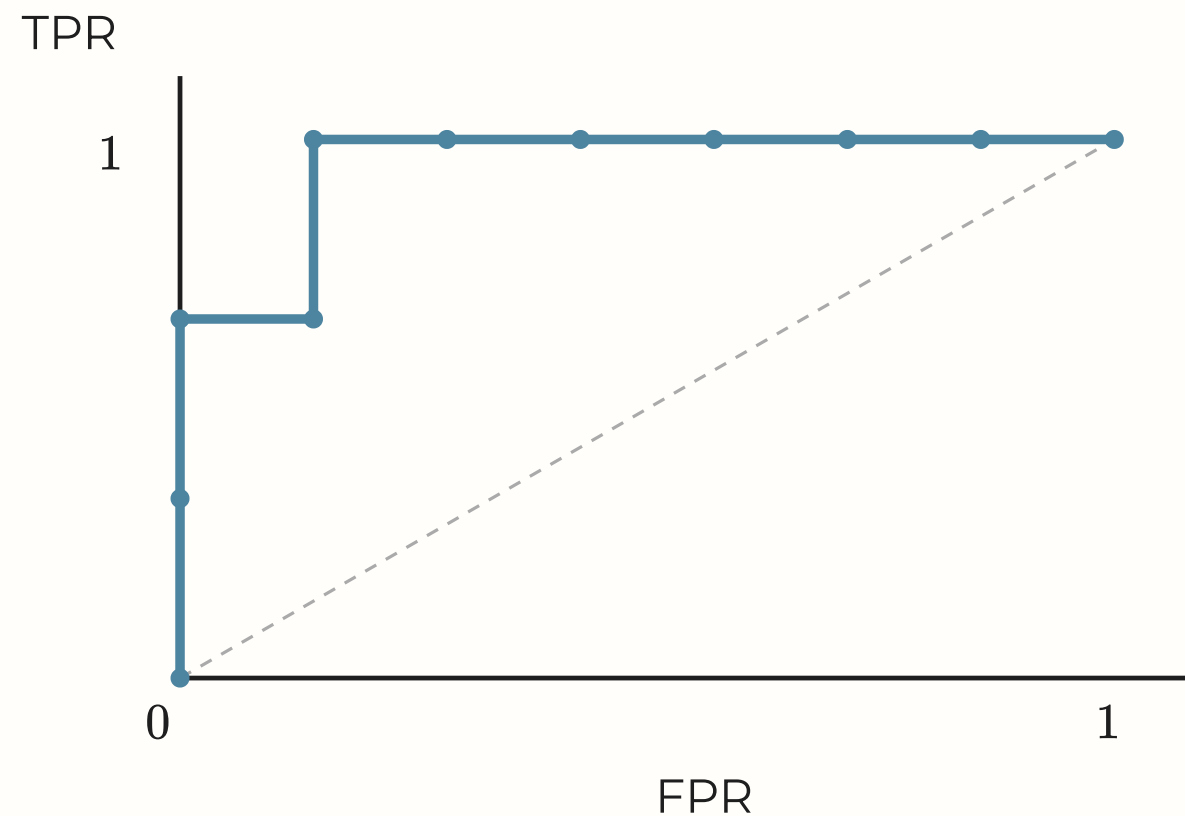
$$\text{TPR} = \frac{TP}{TP + FN} = \text{Recall}$$

$$\text{FPR} = \frac{FP}{FP + TN}$$

Уменьшать порог – по очереди включать объекты в положительный класс.

Для ROC нужны скоры; калиброванные вероятности не обязательны.

ROC-AUC проверяет порядок пар



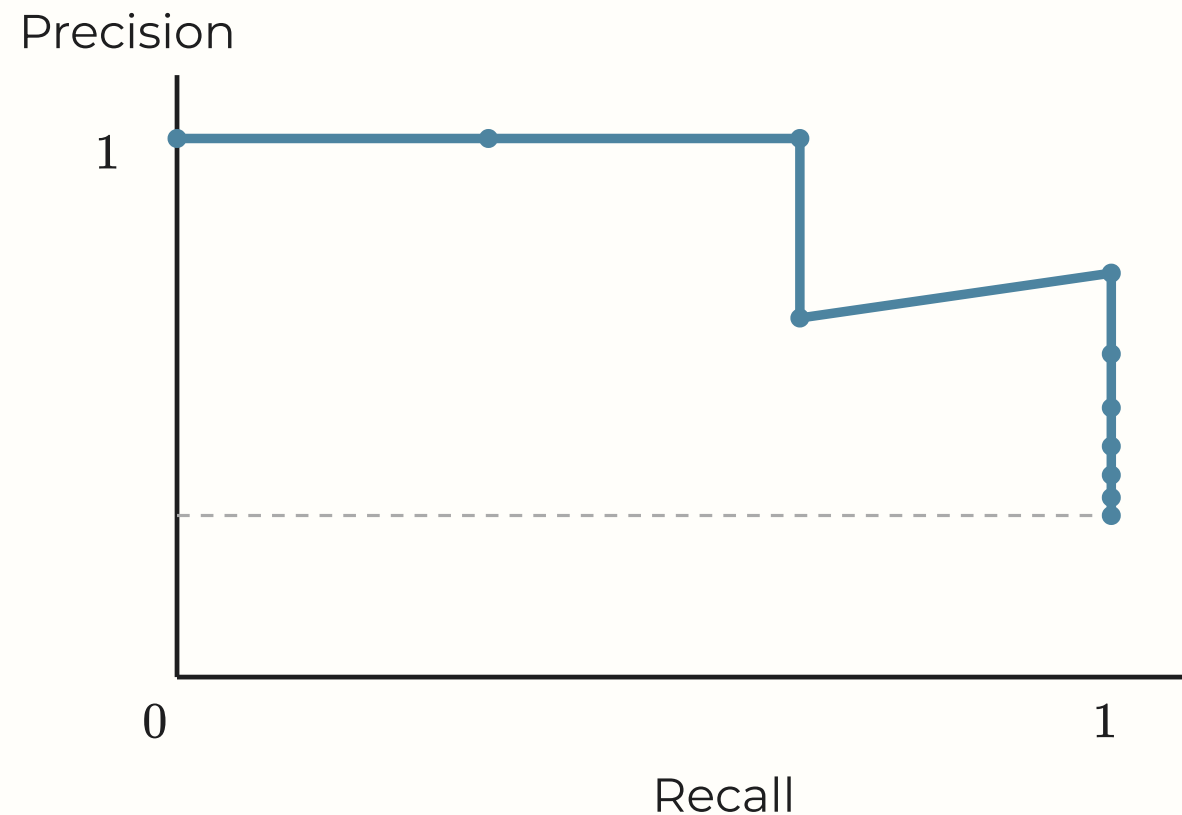
$$\text{AUC} = P(s_+ > s_-) + \frac{1}{2}P(s_+ = s_-)$$

$$\text{AUC} = \frac{20}{21} \approx 0,952$$

Выше скор у положительного объекта – правильно упорядоченная пара.

Строго возрастающее преобразование скоров сохраняет AUC.

PR-кривая делает положительный класс центром внимания



Каждая точка отвечает: сколько найдено и сколько среди найденных верно.

При редком положительном классе ложные срабатывания особенно заметны.

Пунктир: доля положительных объектов $3/10$ в нашем наборе.

Average Precision и площадь трапециями под PR-кривой – разные вычисления.

Хорошее ранжирование не гарантирует честные вероятности

Сохранить порядок скоров, но объявить все большие значения почти единицей.

Порядок пар тот же



ROC-AUC тот же



Вероятностная ошибка
может измениться

Калибровка: среди объектов с прогнозом $p \approx 0,8$ доля положительных должна быть около 0,8.

Это отдельная проверка на данных, не участвовавших в подгонке калибровки.

Если классов больше двух

Пусть C – число классов. Каждый класс рассматривать как «этот класс против остальных».

$$F_{1,\text{macro}} = \frac{1}{C} \sum_{c=1}^C F_{1,c}$$

Macro: всем классам равный вес. Micro: сначала сложить TP, FP и FN всех классов.

Weighted: усреднить показатели с весами по числу истинных объектов класса.

Редкий класс может потеряться в среднем; смотреть и метрики каждого класса.

Что именно нужно выбрать до теста?

Цель – находить положительные объекты, сохраняя заданную долю верных срабатываний.

Что выбирать: число компонент, модель, порог или всё сразу?

Всю цепочку оценивать вместе на валидации/CV.

Например: максимизировать Recall при Precision не ниже заданного требования.

Представление, выбор и проверка – одна цепочка

SVD: оси и
масштабы



PCA: сжатие
разброса



CV: выбрать всю
цепочку



Метрика: оценить
задачу

PCA не видит меток. CV не исправляет утечку автоматически. AUC не выбирает порог и не проверяет калибровку.

Проверка себя: где оценивать центр, сколько направлений оставить и какую ошибку считать важной?